

TABLE OF CONTENTS

Sl. No.	Contents	Page no.
1)	Introduction	01
2)	Methodology with block diagram	01-03
3)	Components used	04
4)	Algorithm	05-07
5)	Program	08-09
6)	Result	10-11
7)	Advantages	12
8)	Applications	13
9)	Images of final project	14-16

INTRODUCTION

AUTOMATIC HEAT DETECTOR

Fire accidents can cause serious damage to life and property. To reduce the risk, it is important to detect fire at an early stage. One way to do this is by using a **heat detector**, which senses high temperatures.

This project is about creating an **automatic heat detector**. It uses a temperature sensor to check the heat level in the area. When the temperature goes above a set limit, the system turns on a buzzer to alert people.

This project helps in understanding how sensors and basic electronics can be used to improve safety.

METHODOLOGY

This project is designed to detect the presence of fire using an IR flame sensor and automatically activate a water pump through a relay module. Additionally, it uses a buzzer for sound alerts and an LCD display for visual updates.

Step-by-Step Working Process:

1) Flame Detection

The IR flame sensor detects infrared light from a flame. It has two outputs:

- D0 (Digital Output): Connected to Arduino pin D3, sends a HIGH or LOW signal depending on flame presence.
- The sensor is powered by 5V and GND from the Arduino.

2) Signal Processing

- The Arduino Uno constantly monitors the D3 pin : If a flame is detected (digital signal changes), it processes this signal to take necessary action.

3) Alarm Activation

- When a flame is detected, the Arduino activates a buzzer connected to D9 to give an audible alert.
- The buzzer's long leg goes to D9, and the short leg to GND.

4) Water Pump Control

- The Arduino sends a signal to the relay module via pin D4.

- The relay is connected to a water pump, using its NO (Normally Open) and COM (Common) terminals.
- When activated, the relay closes the circuit, powering the pump to spray water on the fire.

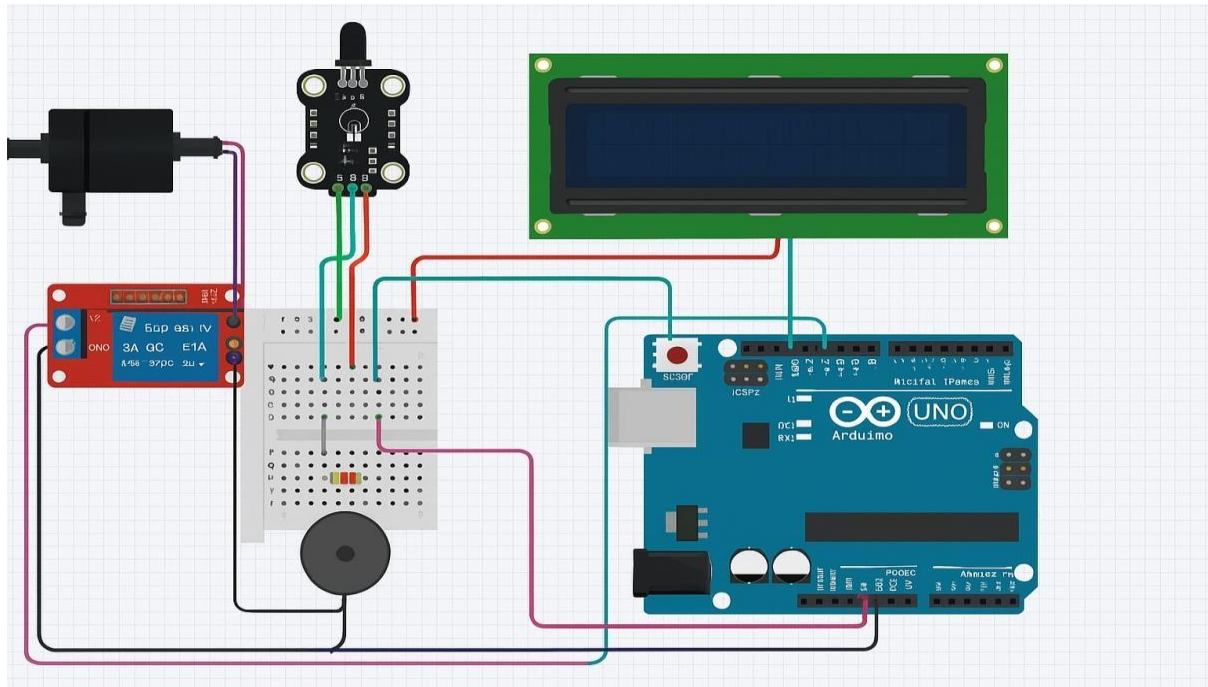
5) Status Display

- An I2C LCD display shows real-time messages like “Fire Detected!” or “System Normal.”
- The LCD is connected using the I2C interface:
 - i. SDA → Arduino SDA
 - ii. SCL → Arduino SCL
 - iii. VCC → 5V
 - iv. GND → GND

6) Power Supply

- All modules are powered by the Arduino’s 5V and GND pins.
- Ensure the pump has a separate power source if it requires higher voltage or current than the Arduino can supply.

BLOCK DIAGRAM:



Working:

- When flame is detected:
- Relay turns ON the pump.
- Buzzer alerts with sound.
- LCD displays "Fire Detected!"
- When no flame is detected:
- Pump and buzzer stay OFF.
- LCD shows system is normal.

COMPONENTS USED

Component	Description	Purpose
IR Flame sensor	A sensor that detects infrared light from a flame	Detects the presence of fire by sensing infrared radiation
Arduino uno	A microcontroller board with digital and analog input/output pins.	Processes sensor data and controls relay, buzzer, and pump
Relay module	An electronic switch that controls high-power devices like a pump	Acts as a switch to control the water pump based on Arduino signal
Buzzer	A small sound-emitting device.	Provides an audible alert when fire is detected.
LCD Display	A display screen that shows information (e.g., temperature or status)	Displays real-time information like "Fire Detected" or "Normal"
Water pump	A pump used to spray water or extinguish the detected fire	Activated by relay to spray water on the fire
Jumper wire	Electrical wires used to make connections between components	Used to make electrical connections between Arduino and other components.
5V power battery	A power source to provide 5V DC to Arduino and other components.	Powers the Arduino and low-power components like the sensor
Power supply (for pump)	A higher voltage power supply (e.g., 12V or 24V) required for the pump.	Provides sufficient power for the water pump.

ALGORITHM OF THE CODE

1. Initialization (Setup Phase) : Start the system.
 - i. Initialize the LCD display with I2C communication.

Set the pin modes:

 - Flame sensor pin as INPUT.
 - Relay pin as OUTPUT.
 - Buzzer pin as OUTPUT.
 - ii. Turn OFF the relay (default safe state).
 - iii. Turn OFF the buzzer.
 - iv. Display the message "SYSTEM READY" and "FUNCTIONING..." on the LCD for 2 seconds.
 - v. Clear the LCD screen.

```
1  #include <LiquidCrystal_I2C.h>
2
3  // Define I2C address for the LCD (0x27, or change based on your setup)
4  LiquidCrystal_I2C lcd(0x27, 16, 2);
5
6  // Define the pins
7  #define SENSOR_PIN 2      // Flame sensor connected to pin 2
8  #define BUZZER_PIN 9      // Buzzer connected to pin 9
9  #define RELAY_PIN 4       // Relay connected to pin 4
10
11 // Variable to store previous state for LCD
12 String previousMessage = "";
13
14 void setup() {
15     // Initialize LCD
16     lcd.init();
17     lcd.clear();
18     lcd.backlight();
19
20     // Set the pin modes
21     pinMode(RELAY_PIN, OUTPUT); // Relay
22     pinMode(SENSOR_PIN, INPUT); // Flame sensor
23     pinMode(BUZZER_PIN, OUTPUT); // Buzzer
24
25     // Initially, ensure relay is off (NO contact)
26     digitalWrite(RELAY_PIN, HIGH); // Relay off
27     digitalWrite(BUZZER_PIN, LOW); // Buzzer off initially
28
29     // Start with a message
30     lcd.setCursor(2, 0);
31     lcd.print("SYSTEM READY");
32     lcd.setCursor(0, 1);
33     lcd.print("  FUNCTIONING...");
34     delay(2000); // Display initial message for 2 seconds
35     lcd.clear();
36 }
```

2. Continuous Monitoring (Loop Phase)

- Read the flame sensor value (sensor Value):
- LOW means flame detected.
- HIGH means no flame detected.
- Print the sensor value to the Serial Monitor for debugging.

```
38 void loop() {  
39     int sensorValue = digitalRead(SENSOR_PIN); // Read the flame sensor value  
40  
41     // Debugging output in the Serial Monitor  
42     Serial.print("Flame Sensor Value: ");  
43     Serial.println(sensorValue);  
44  
45     // If flame is detected (sensor reads LOW)  
46     if (sensorValue == LOW) {  
47         // Fire detected  
48         digitalWrite(RELAY_PIN, LOW); // Activate relay (turn on pump/sprinkler)  
49         digitalWrite(BUZZER_PIN, HIGH); // Activate buzzer (alert the user)
```

3. If Flame is Detected (sensor Value == LOW):

- Activate relay (turn on sprinkler or pump).
- Activate buzzer (sound alarm).
- If the last displayed message is not "FIRE DETECTED!":
- Clear the LCD.
- Display:
 - Line 1: "FIRE DETECTED!"
 - Line 2: "SAFETY PROTOCOL ON"
- Update previous Message to "FIRE DETECTED!".

```
45     // If flame is detected (sensor reads LOW)  
46     if (sensorValue == LOW) {  
47         // Fire detected  
48         digitalWrite(RELAY_PIN, LOW); // Activate relay (turn on pump/sprinkler)  
49         digitalWrite(BUZZER_PIN, HIGH); // Activate buzzer (alert the user)  
50  
51         // Check if the message has changed, and only update LCD if necessary  
52         if (previousMessage != "FIRE DETECTED !") {  
53             lcd.clear(); // Clear the LCD only if the message changes  
54             lcd.setCursor(0, 0);  
55             lcd.print("FIRE DETECTED !");  
56             lcd.setCursor(0, 1);  
57             lcd.print("SAFETY PROTOCOL ON ");  
58             previousMessage = "FIRE DETECTED !"; // Update the previous message  
59         }  
60     }
```

4. If No Flame is Detected (sensor Value == HIGH):

- Deactivate relay (turn off sprinkler or pump).
- Deactivate buzzer.
- If the last displayed message is not "REGULATED TEMP.":
- Clear the LCD.
- Display:
 - Line 1: "REGULATED TEMP."
 - Line 2: "SYESTEM NORMAL" (*Note: "SYESTEM" is a typo, should be "SYSTEM"*)
- Update previous Message to "REGULATED TEMP.".

```
61     else {
62         // No flame detected
63         digitalWrite(RELAY_PIN, HIGH); // Deactivate relay (turn off pump/sprinkler)
64         digitalWrite(BUZZER_PIN, LOW); // Deactivate buzzer (no fire detected)
65
66         // Check if the message has changed, and only update LCD if necessary
67         if (previousMessage != "REGULATED TEMP.") {
68             lcd.clear(); // Clear the LCD only if the message changes
69             lcd.setCursor(0, 0);
70             lcd.print("REGULATED TEMP.");
71             lcd.setCursor(0, 1);
72             lcd.print("SYESTEM NORMAL");
73             previousMessage = "REGULATED TEMP."; // Update the previous message
74         }
75     }
```

5. Wait for 50 milliseconds

- This prevents rapid switching and LCD flickering.

```
77     delay(50); // Small delay to avoid rapid switching and flickering of LCD text
78 }
```

PROGRAM

```
#include <LiquidCrystal_I2C.h>

// Define I2C address for the LCD (0x27, or change based on your setup)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Define the pins
#define SENSOR_PIN 2    // Flame sensor connected to pin 2
#define BUZZER_PIN 9    // Buzzer connected to pin 9
#define RELAY_PIN 4     // Relay connected to pin 4

// Variable to store previous state for LCD
String previousMessage = "";

void setup() {
  // Initialize LCD
  lcd.init();
  lcd.clear();
  lcd.backlight();

  // Set the pin modes
  pinMode(RELAY_PIN, OUTPUT); // Relay
  pinMode(SENSOR_PIN, INPUT); // Flame sensor
  pinMode(BUZZER_PIN, OUTPUT); // Buzzer

  // Initially, ensure relay is off (NO contact)
  digitalWrite(RELAY_PIN, HIGH); // Relay off
  digitalWrite(BUZZER_PIN, LOW); // Buzzer off initially

  // Start with a message
  lcd.setCursor(2, 0);
  lcd.print("System Ready");
  lcd.setCursor(0, 1);
  lcd.print("Awaiting Flame...");
  delay(2000); // Display initial message for 2 seconds
  lcd.clear();
}

void loop() {
  int sensorValue = digitalRead(SENSOR_PIN); // Read the flame sensor value

  // Debugging output in the Serial Monitor
  Serial.print("Flame Sensor Value: ");
  Serial.println(sensorValue);
```

```

// If flame is detected (sensor reads LOW)
if (sensorValue == LOW) {
  // Fire detected

  digitalWrite(RELAY_PIN, LOW); // Activate relay (turn on pump/sprinkler)
  digitalWrite(BUZZER_PIN, HIGH); // Activate buzzer (alert the user)

  // Check if the message has changed, and only update LCD if necessary
  if (previousMessage != "Fire Detected!") {
    lcd.clear(); // Clear the LCD only if the message
    changes    lcd.setCursor(0, 0);    lcd.print("Fire
    Detected!");    lcd.setCursor(0, 1);
    lcd.print("Relay and Buzzer ON");
    previousMessage = "Fire Detected!"; // Update the previous message
  } }
else {
  // No flame detected

  digitalWrite(RELAY_PIN, HIGH); // Deactivate relay (turn off pump/sprinkler)
  digitalWrite(BUZZER_PIN, LOW); // Deactivate buzzer (no fire detected)

  // Check if the message has changed, and only update LCD if
  necessary    if (previousMessage != "Regulated Temp.") {    lcd.clear();
  // Clear the LCD only if the message changes    lcd.setCursor(0, 0);
  lcd.print("Regulated Temp.");    lcd.setCursor(0, 1);
  lcd.print("System Normal");
  previousMessage = "Regulated Temp."; // Update the previous message
  }
}

delay(50); // Small delay to avoid rapid switching and flickering of LCD text

```

RESULT

The IR Flame Sensor-Based Fire Detection and Control System was successfully implemented with the following results:

1. Accurate Fire Detection:

- The IR Flame Sensor accurately detected the infrared radiation emitted by flames. It was able to respond to various flame sources, providing reliable fire detection.

2. Immediate Alert System:

- Upon detecting a fire, the buzzer was activated, providing an immediate audible warning. This serves as a crucial alert mechanism to notify individuals about a potential fire in the area.

3. Water Pump Activation:

- The relay module successfully controlled the water pump. When a fire was detected, the relay switched on the pump, allowing it to spray water and potentially suppress the fire. The pump operated effectively, demonstrating the system's ability to suppress a fire in its early stages.

4. Real-time Monitoring:

- The LCD display provided real-time feedback on the system's status. It displayed messages such as “Fire Detected!” when the sensor triggered and “System Normal” when no fire was present. This allowed users to monitor the system easily.

5. System Efficiency:

- The Arduino processed the signals from the IR Flame Sensor efficiently and activated the appropriate outputs (buzzer, pump) without delays. The system worked seamlessly and responded immediately when fire was detected.

6. Low Power Consumption:

- The system was designed to use low power components, which helped in maintaining energy efficiency, especially during long-term use in fire-prone environments.

7. Reliable Component Integration:

- All components, including the flame sensor, Arduino, relay module, buzzer, LCD display, and water pump, were properly integrated and worked together efficiently. The correct wiring and code ensured smooth operation of the entire system.

8. Easy to Implement and Cost-Effective:

- The system was easy to assemble using affordable components, making it a cost effective fire detection solution. It is suitable for various small-scale applications such as home safety and small workshops.

9. Potential for Future Improvements:

- Future upgrades could include wireless connectivity for remote monitoring and notifications, integration with additional sensors, or the addition of more advanced fire suppression methods.

ADVANTAGES

1. Early Fire Detection:

The system can quickly detect a flame at an early stage, helping to prevent the spread of fire and reduce damage.

2. Automatic Response:

It activates the water pump and buzzer automatically, without requiring human intervention, which saves time in emergencies.

3. Cost-Effective:

The system uses low-cost components like Arduino, flame sensors, and a relay module, making it affordable for small-scale applications.

4. Simple Design:

The circuit is easy to build and understand, making it suitable for educational purposes and DIY home safety projects.

5. Low Power Consumption:

Most components operate on 5V, ensuring energy efficiency and making the system suitable for continuous operation.

6. Real-Time Monitoring:

The I2C LCD display provides live updates about system status, making it easier for users to monitor the environment.

7. Customizable:

The system can be easily expanded with additional sensors (like smoke or gas sensors) or connected to smart systems for notifications.

8. Safe Operation:

The use of a relay ensures electrical isolation between the low-power control side (Arduino) and high-power devices like the pump.

9. Portable and Compact:

The system takes up little space and can be installed in homes, kitchens, or small workshops.

10. Educational Value:

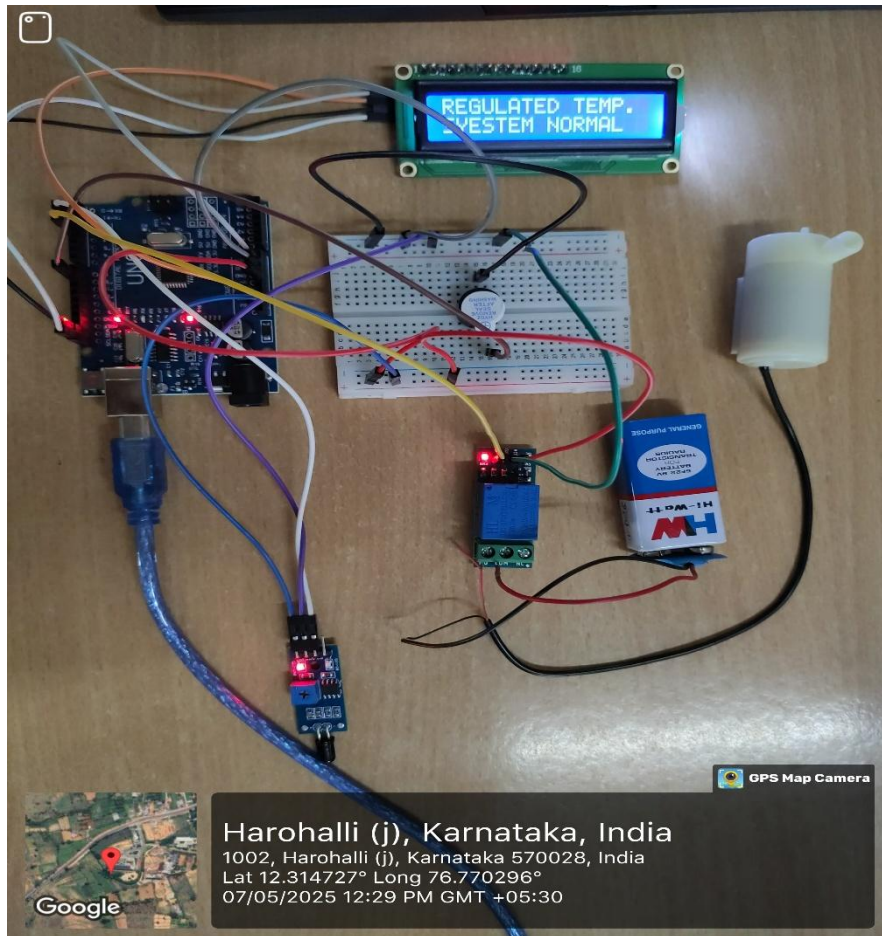
This project helps students and beginners understand practical electronics, sensors, automation, and safety systems.

APPLICATIONS

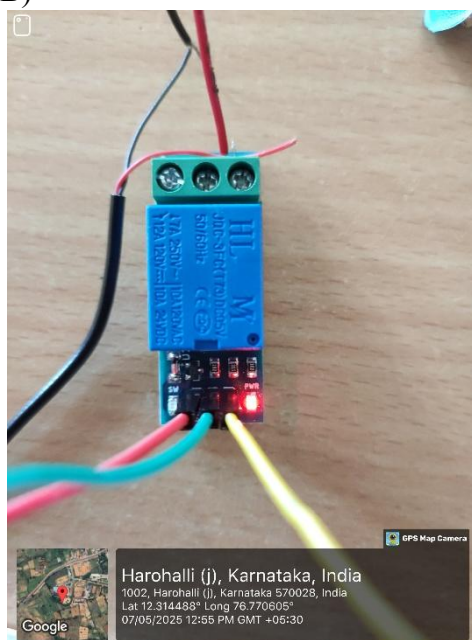
1. **Home Fire Safety:**
Can be installed in kitchens, living rooms, or electrical rooms to detect fire and trigger water spraying in case of emergencies.
2. **Small Offices and Shops:**
Useful in small commercial spaces where early fire detection is crucial to protect goods and equipment.
3. **Laboratories and Workshops:**
Especially suitable in areas with flammable materials or equipment that generates heat.
4. **Warehouses and Storage Units:**
Helps protect stored items from fire damage by detecting flames and activating extinguishing systems.
5. **Schools and Educational Institutions:**
Can be used as a demo project or installed for basic fire safety in labs and classrooms.
6. **Kitchen Safety Systems:**
Ideal for detecting unattended flames or gas stove fires in household or commercial kitchens.
7. **Electronics and Server Rooms:**
Prevents damage to sensitive electronics by detecting fire early and responding quickly.
8. **Smart Home Integration:**
Can be integrated with IoT platforms to notify homeowners remotely about fire incidents.
9. **Remote or Unattended Buildings:**
Useful in areas without regular human presence, like cabins, sheds, or substations.
10. **Prototyping for Advanced Fire Systems:**
Acts as a base model for developing larger, more complex fire safety and automation systems

IMAGES OF FINAL PROJECT:

A)



B)



C)

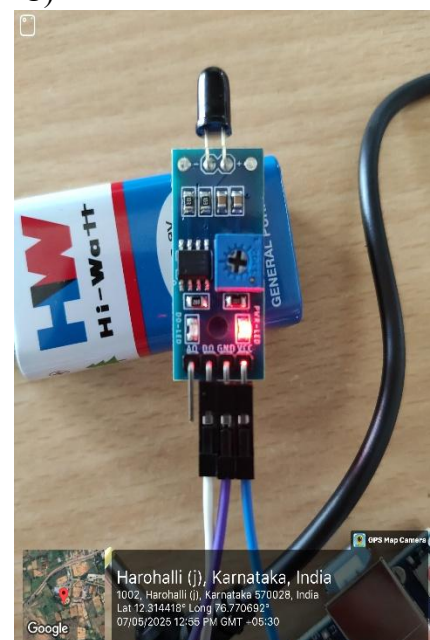
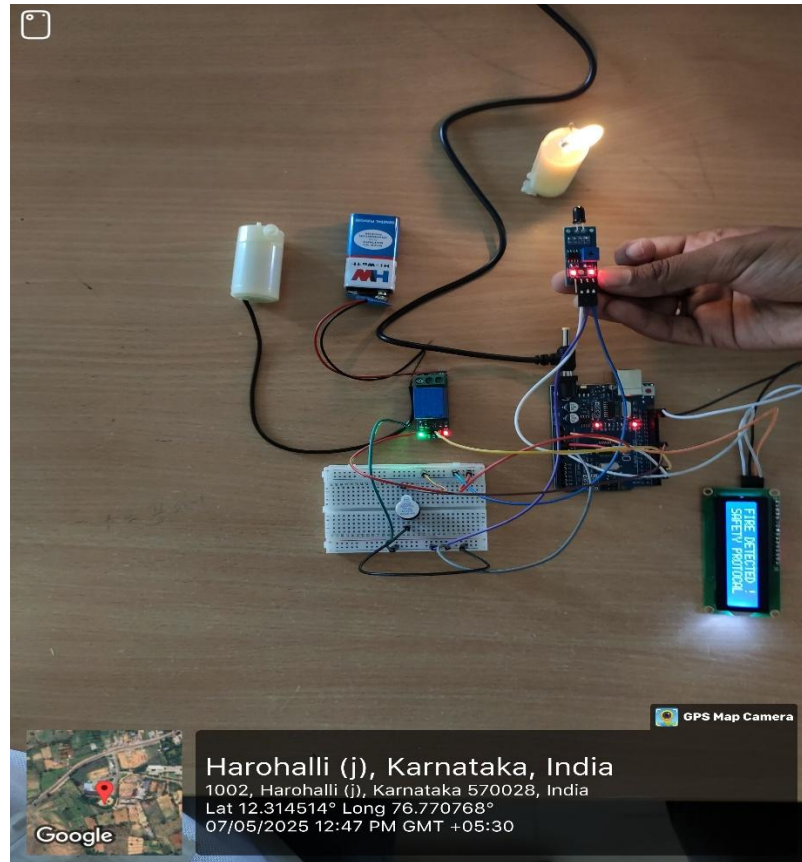
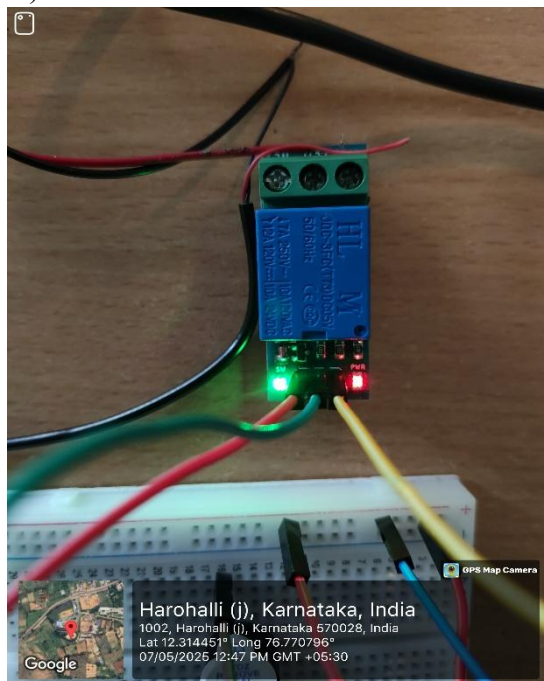


Fig: System before detecting heat. A) overall system, B) Relay module ,C) IR heat sensor

A)



B)



C)

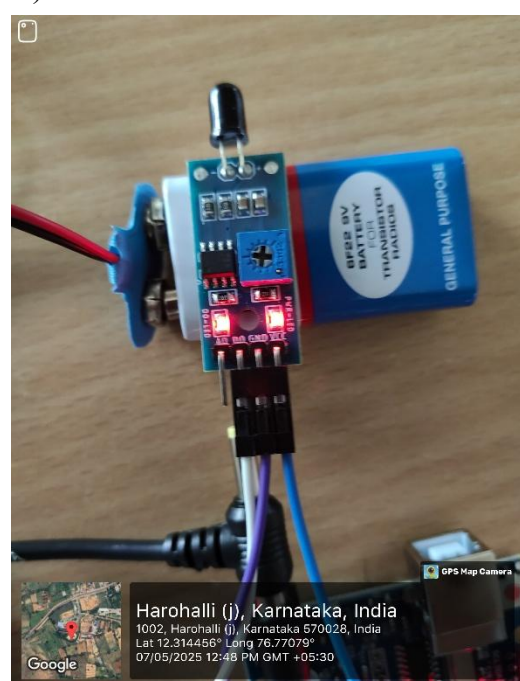


Fig: System after detecting heat. A) overall system, B) Relay module ,C) IR heat sensor



Fig: LCD display message when system is initializing



Fig: LCD display message shown at normal temperature (in the absence of heat)



Fig: LCD display message shown at raised temperature (in the presence of heat)